College of Vocational Studies

University of Delhi

Parallel programming

(Practical File)

Name – Anuradha

Course – BSc(H) CS

College Roll no – 2k22/ CS/11

University Roll no -22013570036

Semester - VI

Index

SNo	Title
1	Implement matrix-matrix multiplication in parallel using OpenMP
2	Implement distributed histogram sorting in parallel using OpenMP
3	Implement breadth first search in parallel using OpenMP
4	Implement Dijkstra's algorithm in parallel using OpenMP
5	Implement Odd-Even Sorting algorithm in parallel using OpenMP

Practicals

1. Implement matrix-matrix multiplication in parallel using OpenMP.

Code -

```
#include <stdio.h>

#include <omp.h>


#define N 3 // Size of matrices


Void multiply_matrices(int A[N][N], int B[N][N], int C[N][N]) {

  #pragma omp parallel for collapse(2)

  For (int I = 0; I < N; i++) {

    For (int j = 0; j < N; j++) {

      C[i][j] = 0;

      For (int k = 0; k < N; k++) {

        C[i][j] += A[i][k] * B[k][j];

      }
```

```
        }
    }
}

Void input_matrix(int mat[N][N], char name) {
    Printf("Enter elements of Matrix %c (%dx%d):\n", name, N, N);
    For (int I = 0; I < N; i++) {
        For (int j = 0; j < N; j++) {
            Printf("%c[%d][%d] = ", name, I, j);
            Scanf("%d", &mat[i][j]);
        }
    }
}

Void print_matrix(int mat[N][N], char name) {
    Printf("Matrix %c:\n", name);
    For (int I = 0; I < N; i++) {
        For (int j = 0; j < N; j++) {
            Printf("%d ", mat[i][j]);
        }
        Printf("\n");
    }
}

Int main() {
    Int A[N][N], B[N][N], C[N][N];
```

```
Int choice;

Printf("Matrix Multiplication Program (using OpenMP)\n");

Input_matrix(A, 'A');
Input_matrix(B, 'B');

Do {
  Printf("\nMenu:\n");
  Printf("1. Multiply matrices\n");
  Printf("2. Display Matrix A\n");
  Printf("3. Display Matrix B\n");
  Printf("4. Display Result Matrix C\n");
  Printf("5. Exit\n");
  Printf("Enter your choice: ");
  Scanf("%d", &choice);

  Switch(choice) {
    Case 1:
      Multiply_matrices(A, B, C);
      Printf("Matrices multiplied successfully!\n");
      Break;
    Case 2:
      Print_matrix(A, 'A');
      Break;
    Case 3:
```

```
            Print_matrix(B, 'B');
            Break;
        Case 4:
            Print_matrix(C, 'C');
            Break;
        Case 5:
            Printf("Exiting program.\n");
            Break;
        Default:
            Printf("Invalid choice! Try again.\n");
    }

  } while (choice != 5);

  Return 0;
}
```

Output –

Matrix Multiplication Program (using OpenMP)

Enter elements of Matrix A (3x3):

A[0][0] = 1

A[0][1] = 2

A[0][2] = 3

A[1][0] = 4

A[1][1] = 5

A[1][2] = 6

A[2][0] = 7

A[2][1] = 8

A[2][2] = 9

Enter elements of Matrix B (3x3):

B[0][0] = 9

B[0][1] = 8

B[0][2] = 7

B[1][0] = 6

B[1][1] = 5

B[1][2] = 4

B[2][0] = 3

B[2][1] = 2

B[2][2] = 1

Menu:

1. Multiply matrices

2. Display Matrix A

3. Display Matrix B

4. Display Result Matrix C

5. Exit

Enter your choice: 1

Matrices multiplied successfully!

Menu:

1. Multiply matrices

2. Display Matrix A

3. Display Matrix B

4. Display Result Matrix C

5. Exit

Enter your choice: 2

Matrix A:

1 2 3

4 5 6

7 8 9

Menu:

1. Multiply matrices

2. Display Matrix A

3. Display Matrix B

4. Display Result Matrix C

5. Exit

Enter your choice: 3

Matrix B:

9 8 7

6 5 4

3 2 1

Menu:

1. Multiply matrices

2. Display Matrix A

3. Display Matrix B

4. Display Result Matrix C

5. Exit

Enter your choice: 4

Matrix C:

30 24 18

84 69 54

138 114 90

Menu:

1. Multiply matrices

2. Display Matrix A

3. Display Matrix B

4. Display Result Matrix C

5. Exit

Enter your choice: 5

Exiting program.

2. Implement distributed histogram sorting in parallel using OpenMP.

Code –

```
#include <stdio.h>

#include <stdlib.h>

#include <omp.h>


#define N 16       // Number of elements

#define MAX_VALUE 9 // Maximum value allowed in array (0 to 9)
```

```
Void histogram_sort(int arr[], int n) {
   Int hist[MAX_VALUE + 1] = {0}; // Histogram array initialized to 0

   // Build histogram in parallel
   #pragma omp parallel for
   For (int I = 0; I < n; i++) {
      #pragma omp atomic
      Hist[arr[i]]++;
   }

   // Rebuild the sorted array from histogram
   Int index = 0;
   For (int I = 0; I <= MAX_VALUE; i++) {
      For (int j = 0; j < hist[i]; j++) {
         Arr[index++] = I;
      }
   }
}

Void input_array(int arr[], int n) {
   Printf("Enter %d elements (values between 0 and %d):\n", n, MAX_VALUE);
   For (int I = 0; I < n; i++) {
      Scanf("%d", &arr[i]);
      If (arr[i] < 0 || arr[i] > MAX_VALUE) {
```

```
        Printf("Invalid input! Value must be between 0 and %d. Please re-enter:\n",
MAX_VALUE);
        i--; // Ask again for the same index
      }
   }
}

Void print_array(int arr[], int n) {
   Printf("Array elements:\n");
   For (int I = 0; I < n; i++)
      Printf("%d ", arr[i]);
   Printf("\n");
}

Int main() {
   Int arr[N];
   Int choice;

   Printf("Histogram Sort Program (using OpenMP)\n");

   Input_array(arr, N);

   Do {
      Printf("\nMenu:\n");
      Printf("1. Sort the array\n");
      Printf("2. Display the array\n");
```

```
    Printf("3. Exit\n");
    Printf("Enter your choice: ");
    Scanf("%d", &choice);

    Switch (choice) {
      Case 1:
        Histogram_sort(arr, N);
        Printf("Array sorted successfully!\n");
        Break;
      Case 2:
        Print_array(arr, N);
        Break;
      Case 3:
        Printf("Exiting program.\n");
        Break;
      Default:
        Printf("Invalid choice! Please try again.\n");
    }
  } while (choice != 3);

  Return 0;
}
```

Output –

Histogram Sort Program (using OpenMP)

Enter 16 elements (values between 0 and 9):

1

3

2

5

7

8

0

1

9

2

4

6

1

3

5

7

Menu:

1. Sort the array

2. Display the array

3. Exit

Enter your choice: 1

Array sorted successfully!

Menu:

1. Sort the array

2. Display the array

3. Exit

Enter your choice: 2

Array elements:

0 1 1 1 2 2 3 3 4 5 5 6 7 7 8 9

Menu:

1. Sort the array

2. Display the array

3. Exit

Enter your choice: 3

Exiting program.

3. Implement breadth first search in parallel using OpenMP.

Code –

```
#include <stdio.h>

#include <stdlib.h>

#include <omp.h>


#define V 6 // Number of vertices (this can be modified)


Void bfs(int graph[V][V], int start) {

   Int visited[V] = {0};

   Int queue[V];

   Int front = 0, rear = 0;
```

```
    Visited[start] = 1;
    Queue[rear++] = start;

    Printf("BFS traversal starting from vertex %d:\n", start);

    While (front < rear) {
        Int u = queue[front++];
        Printf("%d ", u);

        #pragma omp parallel for
        For (int v = 0; v < V; v++) {
            If (graph[u][v] && !visited[v]) {
                #pragma omp critical
                {
                    If (!visited[v]) { // Double check inside critical
                        Visited[v] = 1;
                        Queue[rear++] = v;
                    }
                }
            }
        }
    }
    Printf("\n");
}
```

```
Void input_graph(int graph[V][V]) {
   Printf("Enter the adjacency matrix for the graph (%dx%d):\n", V, V);
   For (int I = 0; I < V; i++) {
      For (int j = 0; j < V; j++) {
         Printf("Edge between vertex %d and %d (0 for no edge, 1 for edge): ", I, j);
         Scanf("%d", &graph[i][j]);
      }
   }
}

Void print_graph(int graph[V][V]) {
   Printf("Adjacency matrix of the graph:\n");
   For (int I = 0; I < V; i++) {
      For (int j = 0; j < V; j++) {
         Printf("%d ", graph[i][j]);
      }
      Printf("\n");
   }
}

Int main() {
   Int graph[V][V];
   Int choice, start_vertex;

   Do {
      Printf("\nMenu:\n");
```

```
Printf("1. Input Graph (Adjacency Matrix)\n");
Printf("2. Display Graph\n");
Printf("3. Perform BFS\n");
Printf("4. Exit\n");
Printf("Enter your choice: ");
Scanf("%d", &choice);

Switch(choice) {
  Case 1:
    Input_graph(graph);
    Break;
  Case 2:
    Print_graph(graph);
    Break;
  Case 3:
    Printf("Enter the starting vertex for BFS (0 to %d): ", V – 1);
    Scanf("%d", &start_vertex);
    Bfs(graph, start_vertex);
    Break;
  Case 4:
    Printf("Exiting program.\n");
    Break;
  Default:
    Printf("Invalid choice! Please try again.\n");
}
```

```
    } while (choice != 4);

    Return 0;
}
```

Output –

Menu:

1. Input Graph (Adjacency Matrix)

2. Display Graph

3. Perform BFS

4. Exit

Enter your choice: 1

Enter the adjacency matrix for the graph (6x6):

Edge between vertex 0 and 0 (0 for no edge, 1 for edge): 0

Edge between vertex 0 and 1 (0 for no edge, 1 for edge): 1

Edge between vertex 0 and 2 (0 for no edge, 1 for edge): 1

Edge between vertex 0 and 3 (0 for no edge, 1 for edge): 0

Edge between vertex 0 and 4 (0 for no edge, 1 for edge): 0

Edge between vertex 0 and 5 (0 for no edge, 1 for edge): 0

Edge between vertex 1 and 0 (0 for no edge, 1 for edge): 1

Edge between vertex 1 and 1 (0 for no edge, 1 for edge): 0

Edge between vertex 1 and 2 (0 for no edge, 1 for edge): 1

Edge between vertex 1 and 3 (0 for no edge, 1 for edge): 1

Edge between vertex 1 and 4 (0 for no edge, 1 for edge): 0

Edge between vertex 1 and 5 (0 for no edge, 1 for edge): 0

Edge between vertex 2 and 0 (0 for no edge, 1 for edge): 1

Edge between vertex 2 and 1 (0 for no edge, 1 for edge): 1

Edge between vertex 2 and 2 (0 for no edge, 1 for edge): 0

Edge between vertex 2 and 3 (0 for no edge, 1 for edge): 0

Edge between vertex 2 and 4 (0 for no edge, 1 for edge): 1

Edge between vertex 2 and 5 (0 for no edge, 1 for edge): 0

Edge between vertex 3 and 0 (0 for no edge, 1 for edge): 0

Edge between vertex 3 and 1 (0 for no edge, 1 for edge): 1

Edge between vertex 3 and 2 (0 for no edge, 1 for edge): 0

Edge between vertex 3 and 3 (0 for no edge, 1 for edge): 0

Edge between vertex 3 and 4 (0 for no edge, 1 for edge): 1

Edge between vertex 3 and 5 (0 for no edge, 1 for edge): 1

Edge between vertex 4 and 0 (0 for no edge, 1 for edge): 0

Edge between vertex 4 and 1 (0 for no edge, 1 for edge): 0

Edge between vertex 4 and 2 (0 for no edge, 1 for edge): 1

Edge between vertex 4 and 3 (0 for no edge, 1 for edge): 1

Edge between vertex 4 and 4 (0 for no edge, 1 for edge): 0

Edge between vertex 4 and 5 (0 for no edge, 1 for edge): 1

Edge between vertex 5 and 0 (0 for no edge, 1 for edge): 0

Edge between vertex 5 and 1 (0 for no edge, 1 for edge): 0

Edge between vertex 5 and 2 (0 for no edge, 1 for edge): 0

Edge between vertex 5 and 3 (0 for no edge, 1 for edge): 1

Edge between vertex 5 and 4 (0 for no edge, 1 for edge): 1

Edge between vertex 5 and 5 (0 for no edge, 1 for edge): 0

Menu:

1. Input Graph (Adjacency Matrix)

2. Display Graph

3. Perform BFS

4. Exit

Enter your choice: 2

Adjacency matrix of the graph:

0 1 1 0 0 0

1 0 1 1 0 0

1 1 0 0 1 0

0 1 0 0 1 1

0 0 1 1 0 1

0 0 0 1 1 0

Menu:

1. Input Graph (Adjacency Matrix)

2. Display Graph

3. Perform BFS

4. Exit

Enter your choice: 3

Enter the starting vertex for BFS (0 to 5): 0

BFS traversal starting from vertex 0:

0 1 2 3 4 5

Menu:

1. Input Graph (Adjacency Matrix)

2. Display Graph

3. Perform BFS

4. Exit

Enter your choice: 4

Exiting program.

4. Implement Dijkstra's algorithm in parallel using OpenMP.

Code –

```
#include <stdio.h>

#include <limits.h>

#include <omp.h>


#define V 5 // Number of vertices


Int minDistance(int dist[], int visited[]) {

  Int min = INT_MAX, min_index = -1;


  #pragma omp parallel

  {

    Int local_min = INT_MAX;

    Int local_index = -1;


    #pragma omp for nowait

    For (int v = 0; v < V; v++) {

      If (!visited[v] && dist[v] < local_min) {

        Local_min = dist[v];

        Local_index = v;
```

```
        }
      }

      #pragma omp critical
      {
        If (local_min < min) {
          Min = local_min;
          Min_index = local_index;
        }
      }
    }
    Return min_index;
}

Void dijkstra(int graph[V][V], int src) {
    Int dist[V];
    Int visited[V];

    // Initialize distances and visited array
    For (int I = 0; I < V; i++) {
      Dist[i] = INT_MAX;
      Visited[i] = 0;
    }
    Dist[src] = 0;

    For (int count = 0; count < V – 1; count++) {
```

```
        Int u = minDistance(dist, visited);
        Visited[u] = 1;

        #pragma omp parallel for
        For (int v = 0; v < V; v++) {
            If (!visited[v] && graph[u][v] && dist[u] != INT_MAX
                && dist[u] + graph[u][v] < dist[v]) {
                Dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    // Print the distances
    Printf("Vertex \t Distance from Source\n");
    For (int I = 0; I < V; i++)
        Printf("%d \t %d\n", I, dist[i]);
}

Void input_graph(int graph[V][V]) {
    Printf("Enter the adjacency matrix for the graph (%dx%d):\n", V, V);
    For (int I = 0; I < V; i++) {
        For (int j = 0; j < V; j++) {
            Printf("Edge between vertex %d and %d (0 for no edge, positive value for edge weight):
", I, j);
            Scanf("%d", &graph[i][j]);
        }
```

```
    }
}

Void print_graph(int graph[V][V]) {
    Printf("Adjacency matrix of the graph:\n");
    For (int I = 0; I < V; i++) {
        For (int j = 0; j < V; j++) {
            Printf("%d ", graph[i][j]);
        }
        Printf("\n");
    }
}

Int main() {
    Int graph[V][V];
    Int choice, start_vertex;

    Do {
        Printf("\nMenu:\n");
        Printf("1. Input Graph (Adjacency Matrix)\n");
        Printf("2. Display Graph\n");
        Printf("3. Perform Dijkstra's Algorithm\n");
        Printf("4. Exit\n");
        Printf("Enter your choice: ");
        Scanf("%d", &choice);
```

```
    Switch(choice) {
      Case 1:
        Input_graph(graph);
        Break;
      Case 2:
        Print_graph(graph);
        Break;
      Case 3:
        Printf("Enter the starting vertex for Dijkstra's Algorithm (0 to %d): ", V – 1);
        Scanf("%d", &start_vertex);
        Dijkstra(graph, start_vertex);
        Break;
      Case 4:
        Printf("Exiting program.\n");
        Break;
      Default:
        Printf("Invalid choice! Please try again.\n");
    }

  } while (choice != 4);

  Return 0;
}
```

Output –

Menu:

1. Input Graph (Adjacency Matrix)

2. Display Graph

3. Perform Dijkstra's Algorithm

4. Exit

Enter your choice: 1

Enter the adjacency matrix for the graph (5x5):

Edge between vertex 0 and 0 (0 for no edge, positive value for edge weight): 0

Edge between vertex 0 and 1 (0 for no edge, positive value for edge weight): 10

Edge between vertex 0 and 2 (0 for no edge, positive value for edge weight): 0

Edge between vertex 0 and 3 (0 for no edge, positive value for edge weight): 0

Edge between vertex 0 and 4 (0 for no edge, positive value for edge weight): 5

Edge between vertex 1 and 0 (0 for no edge, positive value for edge weight): 0

Edge between vertex 1 and 1 (0 for no edge, positive value for edge weight): 0

Edge between vertex 1 and 2 (0 for no edge, positive value for edge weight): 1

Edge between vertex 1 and 3 (0 for no edge, positive value for edge weight): 0

Edge between vertex 1 and 4 (0 for no edge, positive value for edge weight): 2

Edge between vertex 2 and 0 (0 for no edge, positive value for edge weight): 0

Edge between vertex 2 and 1 (0 for no edge, positive value for edge weight): 0

Edge between vertex 2 and 2 (0 for no edge, positive value for edge weight): 0

Edge between vertex 2 and 3 (0 for no edge, positive value for edge weight): 4

Edge between vertex 2 and 4 (0 for no edge, positive value for edge weight): 0

Edge between vertex 3 and 0 (0 for no edge, positive value for edge weight): 7

Edge between vertex 3 and 1 (0 for no edge, positive value for edge weight): 0

Edge between vertex 3 and 2 (0 for no edge, positive value for edge weight): 6

Edge between vertex 3 and 3 (0 for no edge, positive value for edge weight): 0

Edge between vertex 3 and 4 (0 for no edge, positive value for edge weight): 0

Edge between vertex 4 and 0 (0 for no edge, positive value for edge weight): 0

Edge between vertex 4 and 1 (0 for no edge, positive value for edge weight): 3

Edge between vertex 4 and 2 (0 for no edge, positive value for edge weight): 9

Edge between vertex 4 and 3 (0 for no edge, positive value for edge weight): 2

Edge between vertex 4 and 4 (0 for no edge, positive value for edge weight): 0

Menu:

1. Input Graph (Adjacency Matrix)

2. Display Graph

3. Perform Dijkstra's Algorithm

4. Exit

Enter your choice: 2

Adjacency matrix of the graph:

0 10 0 0 5

0 0 1 0 2

0 0 0 4 0

7 0 6 0 0

0 3 9 2 0

Menu:

1. Input Graph (Adjacency Matrix)

2. Display Graph

3. Perform Dijkstra's Algorithm

4. Exit

Enter your choice: 3

Enter the starting vertex for Dijkstra's Algorithm (0 to 4): 0

Vertex	Distance from Source	
0	0	
1	1	10
2	11	
3	3	16
4	5	
5		

Menu:

1. Input Graph (Adjacency Matrix)

2. Display Graph

3. Perform Dijkstra's Algorithm

4. Exit

Enter your choice: 4

Exiting program.

5. Implement odd-even sorting algorithm in parallel using OpenMP.

Code –

```
#include <stdio.h>

#include <omp.h>


Void oddEvenSort(int arr[], int n) {

   Int sorted = 0;


   While (!sorted) {

      Sorted = 1;
```

```
        // Odd phase
        #pragma omp parallel for
        For (int I = 1; I < n – 1; I += 2) {
            If (arr[i] > arr[I + 1]) {
                Int temp = arr[i];
                Arr[i] = arr[I + 1];
                Arr[I + 1] = temp;
                Sorted = 0;
            }
        }

        // Even phase
        #pragma omp parallel for
        For (int I = 0; I < n – 1; I += 2) {
            If (arr[i] > arr[I + 1]) {
                Int temp = arr[i];
                Arr[i] = arr[I + 1];
                Arr[I + 1] = temp;
                Sorted = 0;
            }
        }
    }
}

Int main() {
    Int n, choice;
```

```
Printf("Enter the number of elements: ");
Scanf("%d", &n);

Int arr[n];

Printf("Enter %d elements:\n", n);
For (int I = 0; I < n; i++)
  Scanf("%d", &arr[i]);

Do {
  Printf("\nMenu:\n");
  Printf("1. Sort the array\n");
  Printf("2. Display the array\n");
  Printf("3. Exit\n");
  Printf("Enter your choice: ");
  Scanf("%d", &choice);

  Switch(choice) {
    Case 1:
      oddEvenSort(arr, n);
      printf("Array sorted successfully!\n");
      break;
    case 2:
      printf("Array elements:\n");
      for (int I = 0; I < n; i++)
```

```
            printf("%d ", arr[i]);
        printf("\n");
        break;
      case 3:
        printf("Exiting...\n");
        break;
      default:
        printf("Invalid choice! Please try again.\n");
    }

  } while (choice != 3);

  Return 0;
}
```

Output –

Enter the number of elements: 9

Enter 9 elements:

6 8 5 13 11 2 7 9 4
7

Menu:

1. Sort the array

2. Display the array

3. Exit

Enter your choice: 1

Array sorted successfully!

Menu:

1. Sort the array

2. Display the array

3. Exit

Enter your choice: 2

Array elements:

2 4 5 6 7 8 9 11 13

Menu:

1. Sort the array

2. Display the array

3. Exit

Enter your choice: 3

Exiting...